Client-Side Map Compilation on the Mobile Edge: Architectural Feasibility, Constraints, and System Paradigms

The conceptualization of edge-compiled Geographic Information Systems (GIS) for mobile applications represents a profound paradigm shift from traditional cloud-centric infrastructure. The proposition for the "FreeHike" application—downloading raw OpenStreetMap (OSM) Protocolbuffer Binary Format (.pbf) regional extracts alongside Digital Elevation Model (DEM) GeoTIFFs, and subsequently executing an on-device compilation into optimized alps_basemap.pmtiles and alps_terrain.pmtiles—challenges the fundamental boundaries of contemporary mobile computing. This architecture transfers the intensive extract, transform, and load (ETL) workload from highly scalable cloud compute clusters directly to thermally constrained, battery-powered edge devices.

In 2026, mobile smartphone architectures possess multi-core processors and localized memory capacities that rival desktop workstations from previous decades. However, mobile operating systems employ aggressive, draconian resource governors explicitly designed to protect battery longevity, user interface responsiveness, and hardware integrity. This comprehensive technical report provides an exhaustive analysis of client-side map compilation, evaluating the cross-compilation of heavy GIS tooling, the stringent memory and thermal limits of iOS and Android background execution, flash storage I/O bottlenecks, and the ultimate viability of modern application frameworks to execute this highly experimental compute model.

Cross-Compilation Ecosystems: WebAssembly versus Native Bridging

The primary structural hurdle in client-side map compilation is establishing the execution environment. Heavy GIS compilation tools—such as Tilemaker for vector tiles or Massif for terrain encoding—are fundamentally designed for server or desktop environments where execution threads and system memory are highly available, and operating system-level sandboxing is minimal[1]. Translating this raw computational power to mobile applications forces an architectural choice between compiling to WebAssembly (WASM) for execution within a WebView or utilizing native plugins bridged directly to the application logic.

The WebAssembly and Origin Private File System (OPFS) Mirage

WebAssembly has theoretically democratized heavy computation within the browser environment. When combined with the Origin Private File System (OPFS), WASM applications can bypass traditional IndexedDB storage bottlenecks, gaining byte-level, synchronous read and write access to the local file system via the FileSystemSyncAccessHandle[3]. For an application utilizing the Capacitor framework, deploying a WASM-compiled C++ or Rust tool to execute over OPFS appears, at first glance, to be the most elegant cross-platform strategy.

The OPFS API has proven remarkably performant in benchmark testing; read and write operations execute in the microsecond range, taking up only a single-digit percentage of the overall time spent marshaling data through standard virtual file system APIs[4]. Implementations of SQLite compiled to WebAssembly, such as wa-sqlite, leverage the OPFSCoopSyncVFS or IDBBatchAtomicVFS to achieve synchronous file system operations within dedicated Web Workers, supporting databases exceeding 1GB without catastrophic performance degradation[4].

However, the reality of executing WASM within mobile WebViews presents insurmountable systemic barriers for workloads of planetary or regional map compilation magnitude. First, traditional 32-bit WASM enforces a strict 4GB virtual memory address limit[8]. While the WebAssembly 64-bit standard theoretically allows for 16 exabytes of addressable space, engine implementations (such as V8 in Chromium or JavaScriptCore in WebKit) actively cap WASM64 memory configurations to an absolute maximum of 16GB[10]. Furthermore, browsers achieve memory safety by reserving the entire address space upfront to avoid bounds checking. When a module declares a 4GB maximum, the engine attempts to reserve that entire block[11]. On mobile operating systems, demanding massive contiguous memory reservations from a WebView frequently results in immediate allocation failures.

The SharedArrayBuffer and Cross-Origin Conundrum

More critically, compiling regional map tiles requires heavy, sustained multithreading to parse geometries, encode vectors, and compress outputs concurrently. In WebAssembly, multithreading strictly relies on the implementation of SharedArrayBuffer[8]. Modern browser security protocols, enacted following the discovery of Spectre and Meltdown vulnerabilities, dictate that SharedArrayBuffer is only available in a secure, cross-origin isolated context[13]. This isolation mandates the explicit presence of specific HTTP response headers on the top-level document: Cross-Origin-Embedder-Policy: require-corp and Cross-Origin-Opener-Policy: same-origin[13].

In a Capacitor application, the WebView does not fetch files from a traditional remote server; it serves local application files via custom scheme handlers, such as capacitor://localhost on iOS and Android[16]. WebKit's underlying WKURLSchemeHandler notoriously drops, overrides, or prevents the manual injection of these required custom security headers during the initial application load or during subsequent internal reloads[15]. While developers have attempted elaborate workarounds—such as using Service Workers to intercept fetch requests and append the headers client-side—the implementation is highly fragile and frequently results in the complete failure of the multithreading context[15].

Architecture Paradigm	Memory Constraints	Multithreading Support	I/O Interface	Viability for GIS Compilation
WASM (Capacitor/W	4GB hard limit (32-bit),	Blocked by missing	OPFS (Fast, but strictly	Highly

ebView)	restricted 64-bit engine caps	COOP/COEP headers on local schemes	sandboxed)	Problematic
Native Rust (JSI Bridge)	OS-level physical RAM and virtual limits	Unrestricted native OS threads	Native Filesystem (Direct SSD mapping)	Highly Viable

The React Native JSI and Rust Native Paradigm

Given the structural limitations of WebAssembly inside Capacitor WebViews, the only viable path for client-side map compilation in 2026 relies on executing native binaries directly on the host operating system. The React Native New Architecture, powered by the JavaScript Interface (JSI), has revolutionized cross-platform native execution[18]. JSI entirely removes the asynchronous, JSON-serialized bridge that historically choked React Native performance during heavy data transfers[20]. Instead, JSI allows JavaScript to hold direct memory references to C++ objects, enabling zero-serialization synchronous invocations[19].

By leveraging modern native toolchains such as expo-modules-rs and uniffi, systems engineers can author core GIS logic entirely in Rust, compile it to static libraries for iOS (aarch64-apple-ios) and Android (aarch64-linux-android), and expose type-safe bindings directly to the JavaScript layer[21]. Under this paradigm, the application relies on mature Rust crates to parse raw OSM data and process DEM GeoTIFFs[2]. Because the Rust binary executes natively, it entirely bypasses the WebView's memory sandbox, escapes the SharedArrayBuffer security nightmare, and interacts directly with the mobile device's NVMe or UFS flash storage[25]. The JavaScript user interface acts merely as a presentational layer, dispatching commands to the Rust core and receiving execution state updates via rapid JSI callbacks[23].

Processing Topography: From GeoTIFF to Terrain-RGB

Before analyzing vector tile generation, the compilation of the digital elevation model (DEM) must be addressed. The FreeHike architecture proposes downloading a raw DEM GeoTIFF and compiling it into an alps_terrain.pmtiles archive. Modern 3D map renderers, such as MapLibre GL JS and CesiumJS, expect elevation data to be delivered either as quantized-mesh structures or encoded as RGB raster tiles[27].

Mapbox Terrain-RGB and Mapzen Terrarium are the dominant formats for delivering elevation data to web and mobile clients[27]. These formats encode raw height values in meters into the Red, Green, and Blue channels of standard PNG or WebP images, allowing the client GPU to decode the height on the fly for hillshading or 3D mesh extrusion[28]. For example, the Mapbox Terrain-RGB formula decodes elevation using the equation: height = -10000 + ((R * 256 * 256 + G * 256 + B) * 0.1)[30].

To perform this transformation on the edge, the native Rust core utilizes high-performance geospatial crates such as massif and terrain-codec[2]. The massif crate acts as a rapid Terrain-RGB tile generator, capable of ingesting any GDAL-supported elevation raster and executing multi-threaded processing via the rayon crate to output standard map tiles[2]. The raw GeoTIFF is parsed, reprojected into the Web Mercator projection (EPSG:3857) if necessary, resampled to generate the necessary zoom levels, encoded into WebP images to minimize file size, and subsequently written into a PMTiles archive[2].

While image processing is CPU-intensive, it is highly parallelizable and generally requires less contiguous memory than parsing complex topological relationships found in OSM vector data, making terrain compilation a comparatively straightforward operation on modern mobile SoCs.

Navigating Mobile Memory Governance: Parsing the 700MB PBF

The raw data size of a regional OSM .pbf extract (e.g., covering the Alps) frequently hovers around 700MB[31]. The traditional server-side approach to vector tile generation, exemplified by tools like Planetiler or the default configurations of Tilemaker, involves reading the entire dataset into RAM, constructing a massive object/tile index, flattening spatial relationships, and subsequently emitting the tiles[31]. Planetiler explicitly recommends providing at least 0.5x the .pbf file size in free RAM, while Tilemaker's in-memory node store for a large region can easily consume gigabytes of memory[33].

The OS Execution Governor: Jetsam and LMKD

Attempting to load a 700MB PBF entirely into a mobile application's heap space guarantees immediate, non-negotiable termination by the operating system. Mobile memory management differs radically from desktop environments; mobile operating systems do not utilize a traditional disk-backed swap space (paging file) to alleviate RAM pressure[37]. Instead, they rely on aggressive process termination to ensure system stability.

On iOS, the kernel utilizes the memorystatus subsystem, commonly referred to as "Jetsam"[37]. Even on modern devices equipped with 12GB of physical RAM, the operating system imposes an ActiveHard per-process resident memory limit, frequently capped around 2048MB for foreground applications and drastically lower (often around 50MB to 100MB) for background tasks[39]. When the application exceeds this footprint, the kernel issues an uncatchable SIGKILL signal, registering a per-process-limit or vm-compressor-space-shortage termination event in the device's diagnostic analytics[37].

Similarly, Android 17 has significantly advanced its memory restriction architecture. Prior versions relied heavily on the Low Memory Killer Daemon (LMKD) to terminate cached or background processes primarily during global system memory pressure[43]. Android 17 introduces strict per-app memory limits based on Anonymous Resident Set Size (RSS:anon). If a single application begins hoarding memory—even if the overall system still has gigabytes of free RAM—the Android memory limiter will terminate the process to prevent ecosystem degradation[44]. The system utilizes a TRIGGER_TYPE_OOM anomaly detector, abruptly killing the

process and recording an exit reason of REASON_OTHER[45].

Memory Mapping (mmap) and Clean Page Management

To process a 700MB .pbf file without triggering iOS Jetsam or Android 17 LMKD, the Rust-based compilation engine must absolutely abandon the in-memory object graph model. The architectural solution relies entirely on two technical pillars: memory-mapped files (mmap) and iterative stream parsing.

Memory mapping allows the operating system kernel to map the contents of a file stored on the flash drive directly into the virtual address space of the running process[46]. The critical distinction in mobile OS memory accounting lies in the difference between *clean* pages and *dirty* pages. When the FreeHike application mmaps the 700MB .pbf file in a strictly read-only state, the OS pages the file into physical RAM only as those specific byte offsets are accessed[38]. Because these pages perfectly mirror the immutable data residing on the SSD, they are considered "clean."

On iOS, clean memory-mapped pages do not count toward the Jetsam ActiveHard resident memory limit, because the kernel maintains the ability to instantly discard them under sudden memory pressure without risking any data loss[48]. This mechanism allows applications to effectively interact with data structures vastly larger than their physical RAM quota.

However, if the application modifies those mapped pages, or allocates dynamic variables on the heap to construct a massive indexing hash map, those memory pages become "dirty" (anonymous memory)[48]. Dirty pages must be preserved by the system, contributing directly to the RSS:anon limit and inviting immediate OS termination when limits are breached[44]. To maximize virtual memory limits on iOS, developers must configure the application with the Increased Memory Limit and Extended Virtual Addressing entitlements in Xcode, which expands the allowable bounds for mmap regions before address space fragmentation causes allocation failures[48].

Stream Parsing and Intermediate Storage

Recognizing these constraints, the client-side compiler must utilize highly optimized, low-memory stream parsers. Rust crates such as osmpbf or fast-osmpbf are explicitly designed to iterate through the .pbf file blocks sequentially[24]. As OSM entities (Nodes, Ways, Relations) are processed, the compiler cannot store the generated vector geometries in memory.

Instead, the architecture must employ an on-disk temporary store—conceptually similar to Tilemaker's --store flag or FlatGeobuf's streamable indexing layouts[34]. The compiler must write intermediate geometries into segmented files or localized SQLite databases directly on the flash storage[34]. This stream-and-flush architecture keeps the dirty heap footprint consistently below the 50MB danger zone, completely evading the mobile memory governors while steadily digesting the massive GIS payload.

Surviving Background Execution and Thermal Throttling

Vector tile generation is fundamentally CPU-bound and I/O-intensive. Even with highly optimized native Rust binaries utilizing multiple threads, converting a 700MB PBF into optimized vector tiles on a mobile System-on-Chip (SoC) will take considerable time—ranging from several minutes to potentially over an hour depending on the complexity of the Lua-based tagging profiles and hardware capabilities[31]. It is entirely unreasonable to expect the user to keep the "FreeHike" application actively open and visible in the foreground for this duration. Consequently, the compilation must occur in the background.

Background execution on modern mobile platforms is exceptionally hostile to sustained computational workloads. Both Apple and Google view prolonged background processing as a direct threat to device battery life, thermal stability, and overall user experience, resulting in deeply restrictive execution policies[56].

Asynchronous Windows on iOS and Android

On iOS, background execution is strictly regulated by the BGTaskScheduler framework. The application must register a BGProcessingTask, an API explicitly designed for maintenance work that requires several minutes to complete, such as Core Data cleanup or complex data processing[58]. However, a BGProcessingTask is not a guarantee of continuous execution. The iOS kernel relies on opportunistic scheduling; the task will only fire when the system determines the device is idle, typically connected to external power, and sufficiently cooled[57]. When the task does execute, the window is strictly finite. Empirical testing and developer consensus indicate that iOS will forcefully terminate a BGProcessingTask after approximately 295 seconds (just under 5 minutes), at which point an expiration handler is invoked[60]. If the application does not gracefully halt its operations, save its state, and call endBackgroundTask within this brief expiration window, the application is penalized via process termination, and future background task requests may be severely throttled or entirely denied by the OS[40].

Android's background execution environment is equally treacherous, governed by App Standby Buckets, Doze Mode, and highly aggressive OEM-specific battery optimizers[61]. The standard, modern API for deferred work is WorkManager. However, Android 15+ has introduced stringent foreground service enforcements. Even if the compilation is launched as a Foreground Service—displaying a persistent notification informing the user that map compilation is in progress—workloads categorized under dataSync or mediaProcessing are subject to a strict 6-hour total execution limit per rolling 24-hour window[61]. Furthermore, if the device enters Doze mode (triggered when the screen is off, unplugged, and stationary), CPU wakes are heavily restricted, and continuous computation will be suspended entirely until a designated maintenance window occurs[61].

OS Constraint	Scheduling API	Maximum Continuous Duration	Interruption Triggers
iOS 17+	BGProcessingTask	~295 Seconds (4.9 Minutes)	Expiration handler invoked; OS forces

			suspension
Android 15+	WorkManager / FGS	6 hours (total per 24h window)	Doze Mode, OEM Battery Killers, LMKD

The Idempotent Pipeline Architecture

To survive these fractured, unpredictable execution environments, the client-side map compiler must be architected as a deeply resilient, chunked, and idempotent state machine[23]. The compilation process cannot be structured as a single, monolithic function execution. Instead, the Rust core must persist its exact compilation state to the disk continuously. The workflow must be rigorously segmented:

1. **Phase 1:** Stream parse the PBF via mmap, filter relevant tags (Ways, Nodes, Relations), and write them to intermediate, spatially partitioned disk stores on the flash drive[34].
2. **Phase 2:** Resolve complex geometries, merge overlapping polygons, and snap them to the internal vector tile grid.
3. **Phase 3:** Encode individual tiles using lightweight compression algorithms and append them to the final archive.

If iOS invokes the expiration handler after 4.5 minutes, the Rust core must immediately halt processing, flush all active memory buffers to disk, record the exact byte offset or last processed spatial chunk ID, and yield execution back to the system[40]. When the BGProcessingTask fires again—potentially hours later—the compilation pipeline reads the state checkpoint and resumes exactly where it left off, without duplicating any prior computational effort[63]. This deferrable, resume-capable architecture is the only mathematically viable method to chew through extensive compilation workloads across fractured, hostile background execution windows.

Storage I/O Performance and the Flash Degradation Vector

The final, and perhaps most physically destructive bottleneck in client-side map compilation is flash storage Input/Output (I/O). Generating map tiles entails creating millions of tiny, discrete spatial chunks. In a traditional server architecture, generating an mbtiles file involves executing millions of random INSERT operations into an SQLite database container[1].

The Threat to Mobile NAND Flash

Mobile devices utilize Universal Flash Storage (UFS) or NVMe Solid State Drives. While sequential read and write speeds on these modules are phenomenal, random access patterns—particularly millions of tiny, synchronized database writes—are disastrous for the hardware.

Executing millions of SQLite inserts on mobile flash storage causes severe write amplification. Because NAND flash cannot overwrite data in place, the flash controller must continuously read a larger block of memory, modify it with the new data, and write the entire block to a new location, subsequently erasing the old block[47]. The SQLite Write-Ahead Log (WAL) mechanism, while safer for data integrity, further compounds this I/O overhead on mobile devices, resulting in wasted cycles[65].

This intense, continuous I/O triggers two critical failure modes on the edge. First, it drains the lithium-ion battery at a staggering rate, as the storage controller operates at maximum power draw. Second, the SoC and the storage controller will generate immense internal heat. Because smartphones entirely lack active cooling mechanisms, hitting the device's thermal threshold will cause the operating system to aggressively downclock the CPU (thermal throttling), dragging a procedure that should require 20 minutes into a multi-hour ordeal[61].

PMTiles v3 and Sequential Hilbert Appends

To mitigate rapid flash destruction and thermal runaway, the target output format must not be an SQLite database; it must be PMTiles v3. Unlike a fragmented file directory of millions of individual .pbf tiles or an mbtiles SQLite container, PMTiles is a single-file archive specifically designed for optimized cloud-native rendering and static file hosting[54].

Crucially, for write performance, the PMTiles specification requires the internal directory index to be sorted via a Hilbert space-filling curve[54]. By sorting the generated vector features by their Hilbert tile ID *in memory* (or within intermediate chunked files on disk) before the final encoding phase, the Rust compiler can write the final alps_basemap.pmtiles file entirely sequentially. The compiler utilizes modern Rust crates like oxigdal-pmtiles or pmtiles2 to append compressed tile data directly to the end of the open file stream, completely eliminating random I/O access patterns and bypassing SQLite's journaling and transaction overhead[68]. This sequential write pattern keeps the flash controller largely dormant, prevents severe thermal spikes, and writes the data to the disk at the maximum theoretical speed of the hardware.

The MapLibre Tile (MLT) Standard: Redefining Edge Payload Geometry

Even with sequential writing patterns, the raw volume of generated vector tile data remains a significant liability for battery life and storage capacity. The Mapbox Vector Tile (MVT) standard, while currently ubiquitous across the industry, relies on row-oriented Protocol Buffers and is structurally inefficient at scale[69]. In 2026, the widespread adoption of the MapLibre Tile (MLT) specification provides the ultimate, necessary optimization for making edge compilation practically viable.

Columnar Architectures versus MVT

MLT is a next-generation vector tile format that utilizes a column-oriented Decomposition Storage Model, conceptually similar to massive data analytic formats like Apache Parquet or Apache Arrow[69]. Where MVT stores data row-by-row (one complete feature at a time), MLT organizes data column-by-column—grouping all x-coordinates together, all y-coordinates

together, and all values of a given string attribute together[69].
By isolating data types, MLT unlocks aggressive, lightweight compression techniques that are mathematically impossible in MVT's row-oriented format. The MLT specification utilizes component-wise delta encoding for spatial coordinates, run-length encoding (RLE) for repeating attribute values, dictionary encoding for string columns with low cardinality, and FastPFOR for integer compression[69]. Furthermore, MLT utilizes "Present" bit streams to efficiently encode sparse columns, allowing null values to consume practically zero space[73].

Storage Reduction Analytics

The empirical data supporting MLT is profound. The format yields up to a 3x to 6x reduction in encoded tile size compared to MVT, particularly on dense, data-heavy zoom levels[74]. For the FreeHike application, compiling to MLT rather than MVT means the Rust encoder writes one-third to one-sixth of the total byte volume to the mobile flash storage[69].
This exponential reduction in disk I/O significantly shortens the overall compilation time, substantially lowers the thermal footprint of the storage controller, and directly preserves the user's battery life[74]. The mlt-core Rust crate can be seamlessly integrated into the React Native JSI pipeline, allowing the mobile device to natively encode MLT streams and package them directly into the final PMTiles archive[75].

Verdict and Strategic Recommendations

Returning to the foundational question: Is there any modern framework that makes Client-Side Map Compilation viable in 2026, or is the "Compute Once on Server, Download Anywhere" model still the only realistic path?
From a purely frictionless user experience perspective, the standard cloud-compiled model remains vastly superior. Downloading a pre-compiled 50MB PMTile archive from a highly distributed CDN edge node takes seconds, consumes negligible battery, and requires zero client-side computational overhead. For standard consumer applications where connectivity is assured, server-side compilation is unequivocally the optimal path.
However, the specific "Client-Side Map Compilation" architecture proposed for the "FreeHike" application—designed for scenarios where cloud infrastructure costs must be minimized or where users wish to generate highly customized, granular offline regions—**is technically viable in 2026**. Yet, achieving this viability demands an uncompromising adherence to a highly specific, native-first systems architecture. Any attempt to build this utilizing WebAssembly, Web Workers, and OPFS inside a Capacitor WebView is destined to fail. The web platform's restrictive 4GB/16GB memory limits, the fragility of SharedArrayBuffer headers (COOP/COEP) in local web environments, and the overhead of the JavaScript garbage collector will invariably result in out-of-memory crashes and silent execution failures.
To successfully implement this highly experimental, edge-compute architecture, systems engineering must execute the following strategic roadmap:

1. **Abandon the Web Environment for Core Logic:** The entire compilation pipeline must be authored in Rust. It must be natively compiled for iOS and Android and bridged to the JavaScript UI layer via React Native's JSI (leveraging frameworks like expo-modules-rs).

This grants the compiler direct access to unrestricted multi-threading and entirely escapes the WebView memory sandbox.

2. **Implement Stream Parsing and Memory Mapping:** The Rust compiler cannot load the 700MB .pbf into RAM. It must utilize memory-mapped file reading (mmap) to treat the raw file as clean virtual memory, and rely on stream parsing to emit intermediate geometries to disk, strictly maintaining a dirty memory (RSS:anon) footprint under 50MB to avoid iOS Jetsam and Android 17 LMKD terminations.
3. **Enforce Idempotent Background Scheduling:** The application must utilize BGProcessingTask (iOS) and WorkManager (Android), chunking the compilation process so that it can be suspended immediately when the OS issues an expiration warning, and resumed seamlessly when the device is idle and charging.
4. **Embrace MLT and Sequential PMTiles:** The output must not be an SQLite database. The compiler must sort geometries by Hilbert ID and write sequentially to a single PMTiles v3 archive. Crucially, the vector tiles must be encoded using the new MapLibre Tile (MLT) specification to achieve up to 6x compression ratios, fundamentally reducing the I/O bottleneck and preventing thermal degradation on the mobile flash storage.

Client-side map compilation is no longer theoretical. By leveraging native Rust via JSI, exploiting columnar MLT compression, and strictly respecting the ruthless nature of mobile OS background governors, a mobile device can reliably digest raw GIS data and generate optimized offline maps on the edge.

Works cited

1. Tilemaker - OpenStreetMap Wiki, https://wiki.openstreetmap.org/wiki/Tilemaker
2. massif - crates.io: Rust Package Registry, https://crates.io/crates/massif
3. Origin private file system - Web APIs | MDN, https://developer.mozilla.org/en-US/docs/Web/API/File_System_API/Origin_private_file_system
4. SQLite 3.40 to support browser API, including OPFS #63 - GitHub, https://github.com/rhashimoto/wa-sqlite/discussions/63
5. The Current State Of SQLite Persistence On The Web: May 2026 Update - PowerSync, https://powersync.com/blog/sqlite-persistence-on-the-web
6. Intent to Experiment: Origin Private File System extension: AccessHandle - Google Groups, https://groups.google.com/a/chromium.org/g/blink-dev/c/-FVIvFovd3g
7. How we sped up Notion in the browser with WASM SQLite, https://www.notion.com/blog/how-we-sped-up-notion-in-the-browser-with-wasm-sqlite
8. WebAssembly Limitations | qouteall notes, https://qouteall.fun/qouteall-blog/2025/WebAsembly%20Limitations
9. Wasm, wasmtime, wasmer: hacks for mmap, breaking 4GB limit - Rust Users Forum, https://users.rust-lang.org/t/wasm-wasmtime-wasmer-hacks-for-mmap-breaking-4gb-limit/60114

10. Wasm 3.0 Completed : r/programming - Reddit, https://www.reddit.com/r/programming/comments/1njovoh/wasm_30_completed/
11. wasm64: support memory larger than 16 GB · Issue #1892 · WebAssembly/spec - GitHub, https://github.com/WebAssembly/spec/issues/1892
12. Is Memory64 actually worth using? | SpiderMonkey JavaScript/WebAssembly Engine, https://spidermonkey.dev/blog/2025/01/15/is-memory64-actually-worth-using.html
13. A guide to enable cross-origin isolation | Articles - web.dev, https://web.dev/articles/cross-origin-isolation-guide
14. How to enable cross origin isolation? (the specifics) - Stack Overflow, https://stackoverflow.com/questions/67676725/how-to-enable-cross-origin-isolation-the-specifics
15. Running ServiceWorker in Capacitor App on iOS #5502 - GitHub, https://github.com/ionic-team/capacitor/discussions/5502
16. bug: SharedArrayBuffer support · Issue #6182 · ionic-team/capacitor - GitHub, https://github.com/ionic-team/capacitor/issues/6182
17. Running ServiceWorker in Capacitor App on iOS - Stack Overflow, https://stackoverflow.com/questions/71245695/running-serviceworker-in-capacitor-app-on-ios
18. React Native vs Flutter 2026: Performance & DX Comparison | SharpSkill, https://sharpskill.dev/en/blog/react-native/react-native-vs-flutter-comparison
19. About the New Architecture - React Native, https://reactnative.dev/architecture/landing-page
20. React Native vs Flutter in 2026: Which Should You Choose? | CompleteDigi Blog, https://completedigi.com/blogs/react-native-vs-flutter-2026.html
21. expo-modules-rs — Rust dev tool // Lib.rs, https://lib.rs/crates/expo-modules-rs
22. expo_modules_rs - Rust - Docs.rs, https://docs.rs/expo-modules-rs
23. How I used Rust for the core app logic in React Native - Reddit, https://www.reddit.com/r/reactnative/comments/1sndpz2/how_i_used_rust_for_the_core_app_logic_in_react/
24. fast_osmpbf - Rust - Docs.rs, https://docs.rs/fast-osmpbf
25. Keeping Wallet Math Native: Yield, Notes, and the React Native Rust Bridge | Payy, https://payy.network/blog/keeping-wallet-math-native-yield-notes-and-the-react-native-rust-bridge
26. Integrating Rust with React Native: When and Why It Actually Makes Sense - Medium, https://medium.com/@silverskytechnology/integrating-rust-with-react-native-when-and-why-it-actually-makes-sense-2d28af49a450
27. reearth/reearth-terrain: Open terrain tiles for 3D globes - GitHub, https://github.com/reearth/reearth-terrain
28. Terrain RGB by MapTiler | Guides | Map tiling hosting, https://docs.maptiler.com/guides/map-tiling-hosting/data-hosting/rgb-terrain-by-maptiler/
29. terrain_codec - Rust - Docs.rs, https://docs.rs/terrain-codec

30. Mapbox Terrain-RGB v1 | Tilesets, https://docs.mapbox.com/data/tilesets/reference/mapbox-terrain-rgb-v1/
31. What we learned when a user tried to load a massive GML file in a browser., https://geodataviewer.com/blog/why-vector-tiles-for-large-gis-datasets/
32. Basic guidance on memory requirements · Issue #310 · systemed/tilemaker - GitHub, https://github.com/systemed/tilemaker/issues/310
33. onthegomap/planetiler: Flexible tool to build planet-scale vector tilesets from OpenStreetMap data fast - GitHub, https://github.com/onthegomap/planetiler
34. Scaling Tilemaker to the entire planet, even when it doesn't fit in RAM #593 - GitHub, https://github.com/systemed/tilemaker/discussions/593
35. tilemaker/docs/RUNNING.md at master - GitHub, https://github.com/systemed/tilemaker/blob/master/docs/RUNNING.md
36. Show HN: Tilemaker – DIY vector tiles from OpenStreetMap data | Hacker News, https://news.ycombinator.com/item?id=27781895
37. my app is killed by jetsam but reason is "compressor-space" - Stack Overflow, https://stackoverflow.com/questions/58847790/my-app-is-killed-by-jetsam-but-reason-is-compressor-space
38. Overview of memory management | App quality - Android Developers, https://developer.android.com/topic/performance/memory-overview
39. Safari & Web | Apple Developer Forums, https://developer.apple.com/forums/forums/topics/safari-and-web-topic?page=3
40. Why is my app getting killed? - WWDCNotes, https://wwdcnotes.com/documentation/wwdc20-10078-why-is-my-app-getting-killed/
41. I built an open-source iOS keyboard for rendering LaTeX in chat apps (real-time, native Core Graphics) - Reddit, https://www.reddit.com/r/LaTeX/comments/1ryfje6/i_built_an_opensource_ios_keyboard_for_rendering/
42. Reducing Memory Terminations in iOS Apps | by Alex Cohen - Medium, https://medium.com/@alexandercohen/reducing-memory-terminations-in-ios-apps-3e76797ca5bd
43. Low memory killers | Android Developers, https://developer.android.com/games/optimize/vitals/lmk
44. Android app memory limits in place from Android 17 : r/androiddev - Reddit, https://www.reddit.com/r/androiddev/comments/1ubgnw0/android_app_memory_limits_in_place_from_android_17/
45. Prioritizing Memory Efficiency: Essential Steps for Android 17, https://android-developers.googleblog.com/2026/06/prioritizing-memory-efficiency-steps-for-android-17.html
46. What is the limit on virtual memory for iOS? - Stack Overflow, https://stackoverflow.com/questions/32664679/what-is-the-limit-on-virtual-memory-for-ios
47. Efficient Memory Mapped File I/O for In-Memory File Systems | USENIX, https://www.usenix.org/system/files/conference/hotstorage17/hotstorage17-paper-choi.pdf

48. Two Entitlements to Boost Memory Allocation for iOS Apps - Zenn, https://zenn.dev/mtfum/articles/ios_memory_entitlements?locale=en
49. Gemma 4 E4B .litertlm: max_num_tokens >4096 crashes with SIGSEGV in reshape on iOS arm64 · Issue #6765 · google-ai-edge/LiteRT - GitHub, https://github.com/google-ai-edge/LiteRT/issues/6765
50. mlboydaisuke/Qwen2.5-Omni-3B-Audio-CoreAI - Hugging Face, https://huggingface.co/mlboydaisuke/Qwen2.5-Omni-3B-Audio-CoreAI
51. Making the most of your memory with mmap - KDAB, https://www.kdab.com/making-the-most-of-your-memory-with-mmap/
52. PBF Format - OpenStreetMap Wiki, https://wiki.openstreetmap.org/wiki/PBF_Format
53. Building an OpenStreetMap app in Rust, Part IV - Reddit, https://www.reddit.com/r/rust/comments/lvfk7o/building_an_openstreetmap_app_in_rust_part_iv/
54. Kicking the Tires: Flatgeobuf - Horace Williams, https://worace.works/2022/02/23/kicking-the-tires-flatgeobuf/
55. Tilemaker 3.0 - creating vector tiles - General talk - OpenStreetMap Community Forum, https://community.openstreetmap.org/t/tilemaker-3-0-creating-vector-tiles/108111
56. How to Optimize Background Tasks for Mobile Apps - App Institute, https://appinstitute.com/how-to-optimize-background-tasks-for-mobile-apps/
57. Leveraging the Power of Background Execution (Pt. 2) | by Felipe Ricieri | Medium, https://medium.com/@ricicodes/leveraging-the-power-of-background-execution-pt-2-c637089dc28c
58. Performing long-running tasks on iOS and iPadOS | Apple Developer Documentation, https://developer.apple.com/documentation/backgroundtasks/performing-long-running-tasks-on-ios-and-ipados
59. Handling Background Tasks and Multitasking in iOS Apps - Reintech, https://reintech.io/blog/handling-background-tasks-multitasking-ios-apps
60. `BGAppRefreshTask` vs `BGProcessingTask` understanding timing - Stack Overflow, https://stackoverflow.com/questions/58821689/bgapprefreshtask-vs-bgprocessingtask-understanding-timing
61. Beyond Doze: Building Reliable Background Execution on Modern Android (Including OEM Realities) | by Mustafa Yiğit | ProAndroidDev, https://proandroiddev.com/beyond-doze-building-reliable-background-execution-on-modern-android-including-oem-realities-5fa0a6e05672
62. Android Memory Management : for modern app engineering - Medium, https://medium.com/@kumarpriyansu367/android-memory-management-for-modern-app-engineering-2a493b4d1b2a
63. What I Wish I Knew About Background Tasks on Mobile Apps | by Sanjay Nelagadde, https://levelup.gitconnected.com/what-i-wish-i-knew-about-background-tasks-

on-mobile-apps-db6b18851482
64. Introducing Tippecanoe Directory Support - Geovation Tech Blog, https://geovation.github.io/tippecanoe-directory-support
65. Understanding the SQLite Database - PowerSync Docs, https://docs.powersync.com/maintenance-ops/client-database-diagnostics
66. Easily keep a backend database synced with in-app SQLite for offline-first/local-first Capacitor apps - Reddit, https://www.reddit.com/r/capacitor/comments/1op1cna/easily_keep_a_backend_database_synced_with_inapp/
67. 10 Best iOS Crash Reporting Tools in 2026 - Embrace.io, https://embrace.io/blog/best-ios-crash-reporting-tools/
68. pmtiles2 - Rust - Docs.rs, https://docs.rs/pmtiles2
69. MapLibre Tiles (MLT) • freestiler - WALKER DATA, https://walker-data.com/freestiler/articles/maplibre-tiles.html
70. oxigdal_pmtiles - Rust - Docs.rs, https://docs.rs/oxigdal-pmtiles
71. PMTiles in pmtiles2 - Rust - Docs.rs, https://docs.rs/pmtiles2/latest/pmtiles2/struct.PMTiles.html
72. pmtiles - crates.io: Rust Package Registry, https://crates.io/crates/pmtiles
73. Basics - MapLibre Tile Specification, https://maplibre.org/maplibre-tile-spec/specification/
74. (PDF) MapLibre Tile: A Next Generation Vector Tile Format - ResearchGate, https://www.researchgate.net/publication/394488572_MapLibre_Tile_A_Next_Generation_Vector_Tile_Format
75. mlt-core - crates.io: Rust Package Registry, https://crates.io/crates/mlt-core
76. Serving MLT (MapLibre Tiles) - Martin Tile Server Documentation, https://maplibre.org/martin/using-guides/mlt/
77. Announcing MapLibre Tile: a modern and efficient vector tile format, https://maplibre.org/news/2026-01-23-mlt-release/
78. MapLibre Tile (MLT): Next-Gen Geospatial Data - Emergent Mind, https://www.emergentmind.com/topics/maplibre-tile-mlt-format
79. Implementation Status - MapLibre Tile Specification, https://maplibre.org/maplibre-tile-spec/implementation-status/